COMPILADORES

[Subtítulo del documento]

NOMBRES: Abelwancheng JiangWan,
Rubén Sanz López
CORREOS: Abelwancheng.jiangw@um.es
Ruben.s.l@um.es
CONVOCATORIA: MAYO 2026

Contenido

INTRODUCCIÓN 2
FUNCIONES PRINCIPALES..... 2
ANALIZADOR LÉXICO 3
ANALIZADOR SINTÁCTICO 3
ANALIZADOR SEMÁNTICO 3
MANUAL DE USUARIO PARA LA EJECUCIÓN 4
EJEMPLOS DE FUNCIONAMIENTO..... 4
CONCLUSIONES..... 7

INTRODUCCIÓN

En esta práctica desarrollaremos un compilador miniC para ello usaremos Flex para el análisis léxico y Bison para el análisis sintáctico, traduciremos programas escritor en este lenguaje a un código ensamblador MIPS así incluyendo una tabla de símbolos y la generación de código basado en listas de instrucciones donde nos enfrentaremos por ejemplo a la traducción de estructuras como el if-else o el while , donde lo podremos ejecutar en MARS o SPIM.

FUNCIONES PRINCIPALES

Funcion para declarar variables y constantes enteras:
void declarar_id(char *id, Tipo t): es la encargada de dar de alta a las variables y constantes. Esta función tambien se asegura de comprobar si una funciona ya ha sido declarada con el mismo nombre.

Funcion para declarar las cadenas de texto del programa:
int declarar_str(char *str): funciona igual que el declarar_id pero solo para los textos que ponemos en el print.

Funcion para inicializar registros:
void inicializaRegs() : es la función que indica que todos los registros temporales del procesador están vacíos y listos para que el compilador pueda empezar a compilar el código.

Funcion para obtener registro:
char *obtenerReg(): Esta función sirve para cuando necesitamos realizar una operación aritmética en este caso, lo primero busca un registro que este libre y lo ocupamos.

Funcion para imprimir la lista de simbolos de MIPS:
void imprimirLS(): esta función se encarga de escribir una lista con todas las variables y textos en la sección de datos .data del archivo MIPS,

Funcion para imprimir la sección de código de MIPS:
void imprimirLC(ListaC codigo): Es la que escribe el programa principal .text. recorre toda la lista de instrucciones que hemos generado y lo imprime en el formato correcto en MIPS.

Funcion para liberar registro:
void liberarReg(char *reg): esta funcion se encarga de liberar la memoria cuando terminamos de usar un registro es importante liberarla ya que si no lo hacemos nos quedaríamos sin memoria y no podríamos ejecutar la siguiente operación.

ANALIZADOR LÉXICO

El analizador léxico es el encargado de leer el código de entrada carácter por carácter, donde cada carácter tiene su propia etiqueta llamada “Token”. Que están compuestos por las palabras reservadas, operadores y símbolos, identificadores, cadenas de texto, números y la recuperación de errores en modo pánico. Por ultimo hemos implementado los siguientes variables:

- **yylex**: función de Flex que examina el texto de entrada y lo compara con la siguiente coincidencia con las reglas definidas.
- **yytext**: función de Flex que apunta a la ultima secuencia de caracteres que coincidió con alguna expresión regular
- **yyleng**: variable de tipo entero que almacena la longitud de la cadena yytext
- **yylineno**: variable de Flex que nos ayuda a que nos muestre de manera automática en que línea de archivo de entrada nos encontramos en cada momento.
- **yylval**: variable que almacena el valor del token y se lo pasa al sintáctico.
- **strdup(yytext)**: guardamos un copia de ese texto y se lo pasamos al analizador sintáctico a través dde yyval.cadena.

ANALIZADOR SINTÁCTICO

El analizador sintáctico su objetivo es tomar la secuencia de tokens generada por el analizador léxico y comprobar si dichos tokens tienen una estructura valida de acuerdo a la gramática proporcionada en la práctica de miniC.

Para el desarrollo de análisis sintáctico hemos implementado la herramienta Bison donde este recibe la información léxica pasada por Flex. Este resuelve las ambigüedades gracias al (%left), para la gestión de errores hemos incluido la regla error “;” donde descarta los tokens erróneos hasta encontrar un “; ” y así poder analizar el resto de archivos. También tenemos la función yyerror donde muestra por pantalla el número de fallos sintácticos que ha tenido he indicado la línea del error.

ANALIZADOR SEMÁNTICO

El análisis semántico es la encargada de mostrar si tiene una estructura valida el analizador sintáctico. Para ello comprobara si las variables, constantes y tipos de datos son válidas para la generación de código.

Este analizador semántico cumple todos los requisitos correspondientes entonces crearemos una **Tabla de símbolos** donde se guardara todas las variables, constantes , cadenas de texto a medida que el analizador analiza el código proporcionado.

Una vez hecho la tabla de símbolos pasaremos a la **Lista de código** donde en ella traduciremos las instrucciones a código ensamblador.

Por ultimo para imprimir dicha tabla y código usaremos dos funciones: imprimirLS (lista simbolos) y imprimirLC(lista codigo), y también completando la gramática miniC dada en la práctica.

MANUAL DE USUARIO PARA LA EJECUCIÓN

Podemos ejecutar el programa con;

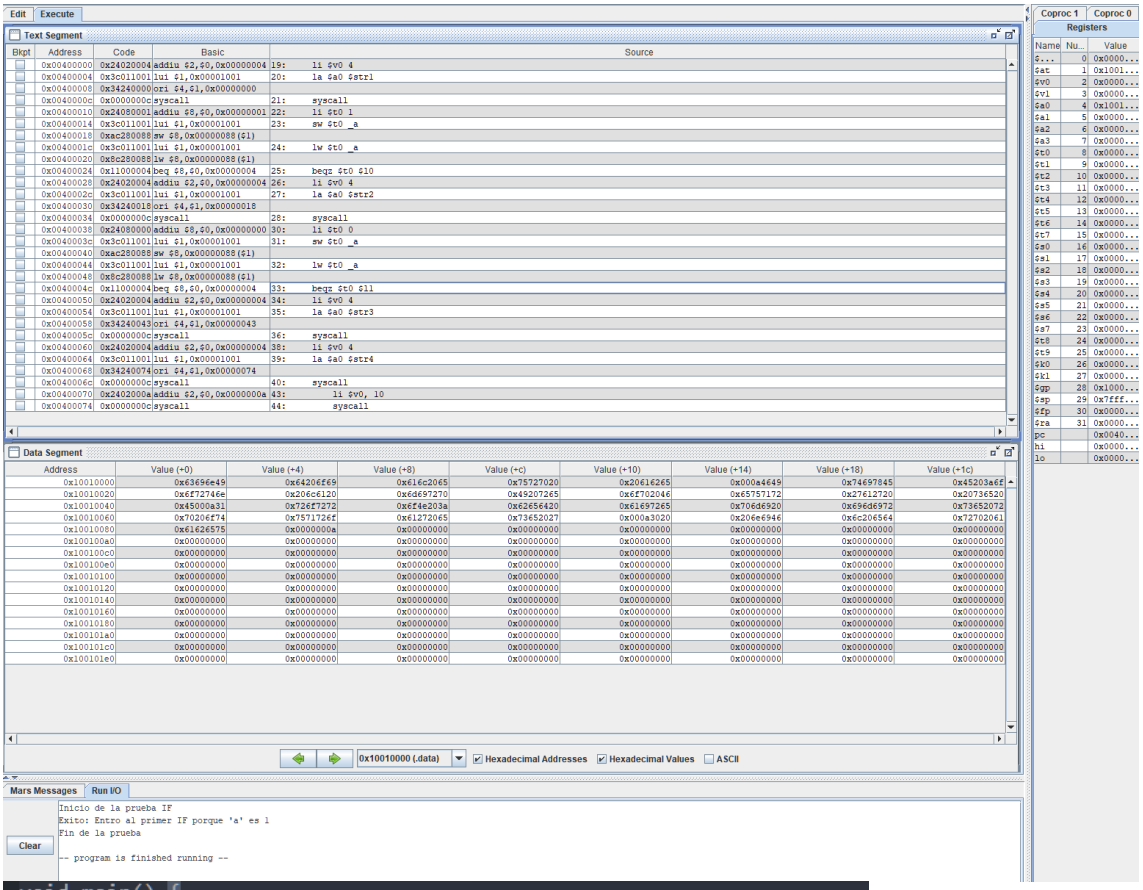
Make run

O podemos ejecutar en el caso de las pruebas para ver si funciona los códigos:

./minic prueba.txt > prueba.s

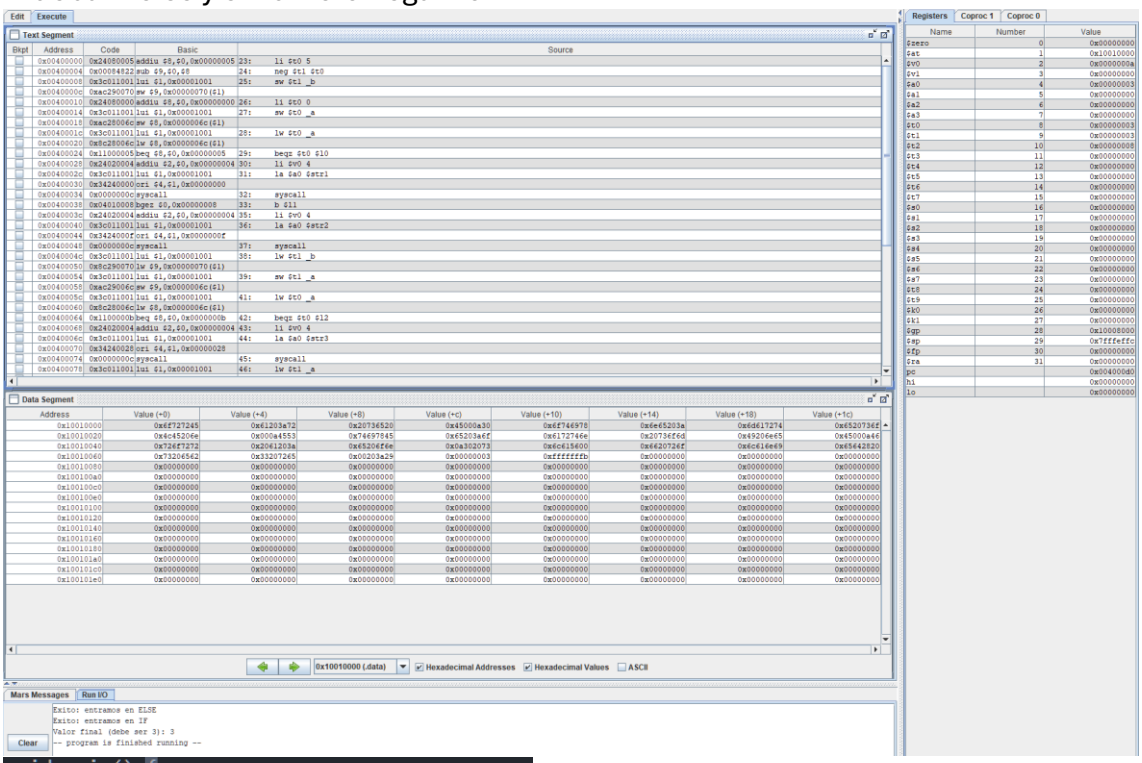
EJEMPLOS DE FUNCIONAMIENTO

Prueba if




```
void main() {
    var int a;
    print("Inicio de la prueba IF\n");
    a = 1;
    if (a) {
        print("Exito: Entro al primer IF porque 'a' es 1\n");
    }
    a = 0;
    if (a) {
        print("Error: No deberia imprimir esto porque 'a' es 0\n");
    }
    print("Fin de la prueba\n");
}
```

Prueba if-else y el numero negativo




```
void main() {
    var int a;
    const int b = -5;

    a = 0;
    if (a) {
        print("Error: a es 0\n");
    } else {
        print("Exito: entramos en ELSE\n");
        a = b; // a ahora vale 5
    }

    if (a) {
        print("Exito: entramos en IF\n");
        a = a + 8; // a ahora vale 3
    } else {
        print("Error: a no es 0\n");
    }

    print("Valor final (debe ser 3): ");
    print(a);
}
```

```
Exito: entramos en ELSE
Exito: entramos en IF
Valor final (debe ser 3): 3
-- program is finished running --
```

Prueba while

Edit | Execute

Text Segment

Bkpt	Address	Code	Basic	Source
	0x00400000	0x24080003	addiu $8,$0,0x00000003	19: li $t0 3
	0x00400004	0x3c011001	lui $1,0x00001001	20: sw $t0 _contador
	0x00400008	0xac28003c	sw $8,0x0000003c($1)	
	0x0040000c	0x24020004	addiu $2,$0,0x00000004	21: li $v0 4
	0x00400010	0x3c011001	lui $1,0x00001001	22: la $a0 $str1
	0x00400014	0x34240000	ori $4,$1,0x00000000	
	0x00400018	0x0000000c	syscall	23: syscall
	0x0040001c	0x3c011001	lui $1,0x00001001	25: lw $t0 _contador
	0x00400020	0x8c28003c	lw $8,0x0000003c($1)	
	0x00400024	0x11000014	beq $8,$0,0x00000014	26: beqz $t0 $l1
	0x00400028	0x24020004	addiu $2,$0,0x00000004	27: li $v0 4
	0x0040002c	0x3c011001	lui $1,0x00001001	28: la $a0 $str2
	0x00400030	0x34240018	ori $4,$1,0x00000018	
	0x00400034	0x0000000c	syscall	29: syscall
	0x00400038	0x3c011001	lui $1,0x00001001	30: lw $t1 _contador
	0x0040003c	0x8c29003c	lw $9,0x0000003c($1)	
	0x00400040	0x24020001	addiu $2,$0,0x00000001	31: li $v0 1
	0x00400044	0x00092021	addu $4,$0,$9	32: move $a0 $t1
	0x00400048	0x0000000c	syscall	33: syscall
	0x0040004c	0x24020004	addiu $2,$0,0x00000004	34: li $v0 4
	0x00400050	0x3c011001	lui $1,0x00001001	35: la $a0 $str3
	0x00400054	0x3424002b	ori $4,$1,0x0000002b	
	0x00400058	0x0000000c	syscall	36: syscall
	0x0040005c	0x3c011001	lui $1,0x00001001	37: lw $t1 _contador
	0x00400060	0x8c29003c	lw $9,0x0000003c($1)	
	0x00400064	0x240a0001	addiu $10,$0,0x0000...	38: li $t2 1
	0x00400068	0x012a4822	sub $9,$9,$10	39: sub $t1 $t1 $t2
	0x0040006c	0x3c011001	lui $1,0x00001001	40: sw $t1 _contador
	0x00400070	0xac29003c	sw $9,0x0000003c($1)	
	0x00400074	0x0401ffe9	bgez $0,0xffffffe9	41: b $l0
	0x00400078	0x24020004	addiu $2,$0,0x00000004	43: li $v0 4

Data Segment

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	0x63696e49	0x64206f69	0x62206c65	0x656c6375	0x69687720	0x000a656c	0x63206c45	0x61746e6f
0x10010020	0x20726f64	0x656c6176	0x0a00203a	0x6e694600	0x6c656420	0x63756220	0x000a656c	0x00000000
0x10010040	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010060	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010080	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100a0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100c0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100e0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010100	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010120	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010140	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010160	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010180	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100101a0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100101c0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100101e0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

Coproc 1 | Coproc 0

Registers

Name	Number	Value
$zero	0	0x00000000
$at	1	0x10010000
$v0	2	0x0000000a
$v1	3	0x00000000
$a0	4	0x1001002d
$a1	5	0x00000000
$a2	6	0x00000000
$a3	7	0x00000000
$t0	8	0x00000000
$t1	9	0x00000000
$t2	10	0x00000001
$t3	11	0x00000000
$t4	12	0x00000000
$t5	13	0x00000000
$t6	14	0x00000000
$t7	15	0x00000000
$s0	16	0x00000000
$s1	17	0x00000000
$s2	18	0x00000000
$s3	19	0x00000000
$s4	20	0x00000000
$s5	21	0x00000000
$s6	22	0x00000000
$s7	23	0x00000000
$t8	24	0x00000000
$t9	25	0x00000000
$k0	26	0x00000000
$k1	27	0x00000000
$gp	28	0x10008000
$sp	29	0x7fffeffc
$fp	30	0x00000000
$ra	31	0x00000000
pc		0x00400090
hi		0x00000000
lo		0x00000000

```
void main() {
    var int contador;
    contador = 3;

    print("Inicio del bucle while\n");

    while (contador) {
        print("El contador vale: ");
        print(contador);
        print("\n");

        contador = contador - 1;
    }

    print("Fin del bucle\n");
}
```

```
Inicio del bucle while
El contador vale: 3
El contador vale: 2
El contador vale: 1
Fin del bucle

-- program is finished running --
```

Prueba de errores léxicos
Prueba de errores sintácticos
Prueba de errores semánticos

```
void main() {
    const int TOPE = 100;
    var int calculo;

    var int TOPE; // error semantico (re-declaración)

    calculo = 50 #@ 2; // error lexico y sintactico

    calculo= 5 @ 2; // error lexico y sintactico

    print( 5 + * 2 ); // error sintactico

    TOPE = 200; // error semantico

    resultadoooo = 10; // error semantico
}
```

```
Error en linea 5: TOPE ya declarado
Error léxico: símbolos no reconocido en la linea 7 #@
Error sintáctico en linea 7: syntax error, unexpected number
Error léxico: símbolo no reconocido en la linea 9 @
Error sintáctico en linea 9: syntax error, unexpected number
Error sintáctico en linea 11: syntax error, unexpected *, expecting - or number or ( or identifier
Error semántico en linea 13: TOPE es una constante
Error semántico en linea 15: resultadoooo no declarado
```

CONCLUSIONES

Este proyecto nos ha servido para aprender más sobre Flex y Bison y poner en práctica los conceptos teóricos sobre la estructura interna de los compiladores, así como sobre el funcionamiento de sus distintas etapas de análisis, la detección y resolución de errores en estas y la generación de un código miniC a un codigo ensamblador.